

Deep Learning

CSE641/ECE555

Lecture 3 | Take your own notes during lectures

Vinayak Abrol <abrol@iiitd.ac.in>

Evolution of Convolutional Neural Networks



INDRAPRASTHA INSTITUTE of
INFORMATION TECHNOLOGY DELHI

Recap: MLP

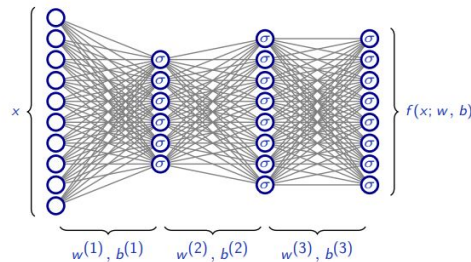
$$W(new) = W(old) + \eta \nabla$$

MLP: Fully connected layer/dense layer, since every input influences every output.

- `torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)`
- weight (torch.Tensor) - shape(out_features,in_features). The values are initialized from uniform distribution $U(-\sqrt{k}, \sqrt{k})$, where $k=1/\text{in_features}$
- bias - shape(out_features)

Issues

- MLPs use one weight for each input dimension
- The number of weights may rapidly becomes unmanageable
- For structured data MLPs often don't capture invariant features
 - E.g., translation invariance in images
- More parameters can lead to overfitting
- MLPs are not inherently designed for sequential data
- Training problems. Depth introduces its own issues.



256×256 RGB image as input would require $(256 \times 256 \times 3)^2 \approx 3.87\text{e}+10$ parameters which corresponds to a memory footprint of (>100Gb)

Recap: MLP

$$W(new) = W(old) + \eta \nabla$$

MLP: Fully connected layer/dense layer, since every input influences every output.

- `torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)`
- weight (`torch.Tensor`) - `shape(out_features, in_features)`. The values are initialized from uniform distribution $U(-\sqrt{k}, \sqrt{k})$, where $k=1/\text{in_features}$

Importance of Minimum/Sufficient Width

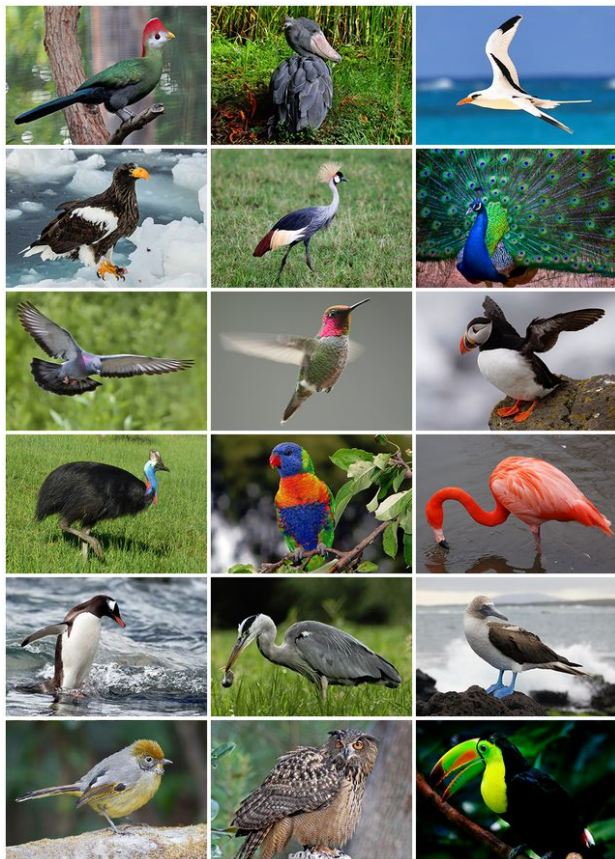
Issues

- Remember the XOR problem
- Consider a deep network with Insufficient Width (1 Neuron per Layer)
- Adding more layers (depth) with a single neuron per layer does not solve the issue because each layer can only compute a linear transformation of the input.
- Hence, we require Single Hidden Layer with Sufficient Width (2 Neurons)

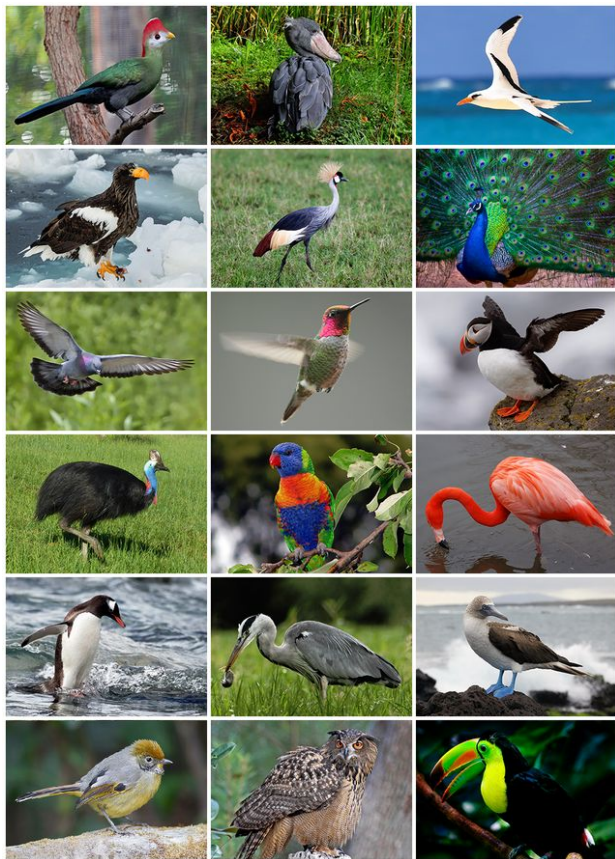
$f(x; w, b)$

256×256 RGB image as input would require $(256 \times 256 \times 3)^2 \approx 3.87e+10$ parameters which corresponds to a memory footprint of (>100Gb)

Convolutional Neural Networks: Motivation



Convolutional Neural Networks: Motivation



What Features to Extract ?

Similarity with neighbor pixels - Semantic Meaning.

Elements of interest can appear anywhere in the image.

CNNs

- So far all we are doing is detecting important (invariant) features in signals, in a very naive way.
- Engineers have been doing this from ages:
 - Images: SIFT, FAST, SURF, BRIEF, etc.
 - Speech: Spectrogram, MFCC, LPCC, etc.
- Issues: these extractors were hand-crafted for small scale tasks and hence either too general or too over-engineered.
- What if we learned to detect these features? [Isn't MLP doing that?]
 - We need a structured way of doing this!

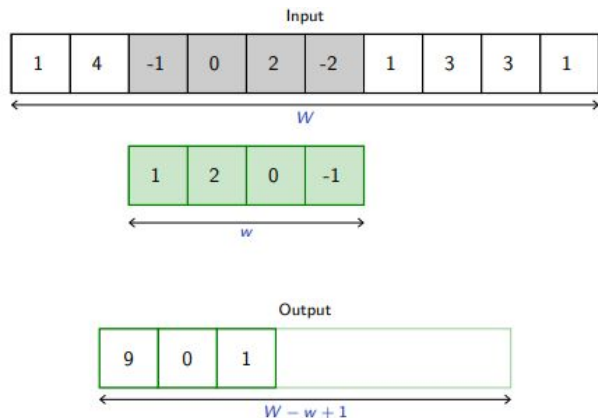
CNNs

- So far all we are doing is detecting **important (invariant) features** in signals, in a very naive way.
- Engineers have been doing this from ages:
 - **Images**: SIFT, FAST, SURF, BRIEF, etc.
 - **Speech**: Spectrogram, MFCC, LPCC, etc.
- Issues: these extractors were hand-crafted for small scale tasks and hence either too **general** or too **over-engineered**.
- What if we learned to detect these features? [Isn't MLP doing that?]
 - We need a structured way of doing this!
- **Convolution** **preserve the structure** of the signal:
 - if the input signal is a 2d tensor, then the output of a convolution layer will be 2d tensor.
 - there will be a clear **relation** between the **locations** of the values in the **input/output** tensors.

CNNs

- So far all we are doing is detecting **important (invariant) features** in signals, in a very naïve way.
- Engineers have been doing this from ages:
 - **Images:** SIFT, FAST, SURF, BRIEF, etc.
 - Generalization of this idea is known as **Representation Learning**.
- **Issues** Inference
 - **either** A process in AI/ML/DL where algorithms are used to extract meaningful patterns from raw data to create representations that are easier to understand and process.
 - **Meaningfulness** is often task dependent and governed by the way we partition the input space. (Remember XOR!)
- **Conclusion**
 - if the input signal is a 2d tensor, then the output of a convolution layer will be 2d tensor.
 - there will be a clear **relation** between the **locations** of the values in the **input/output tensors**.

CNN [Convolution Operator]



Convoluting an input signal with a weight vector also called “kernel” (in green) consists of repeating a dot product between the weight vector and a vector of same length (in gray) extracted from the input vector, for every position of the weight vector while it is swiped across the entire input signal.

Observe convolving a 1D signal with a 1D kernel produces a 1D signal. The resulting tensor is shorter than the input.

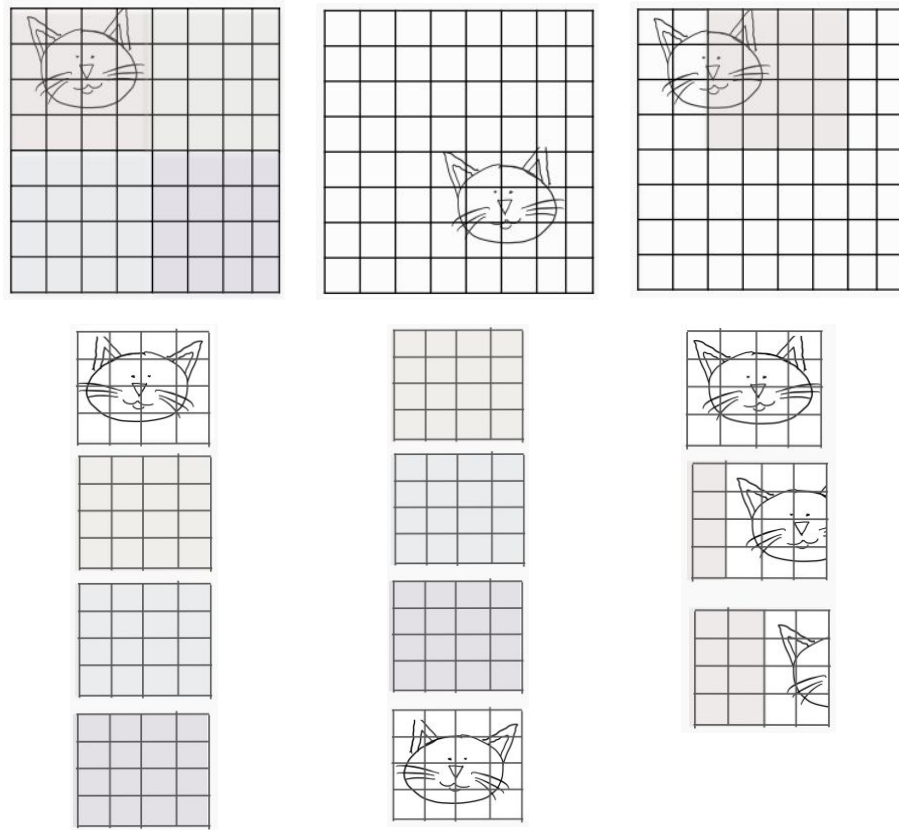
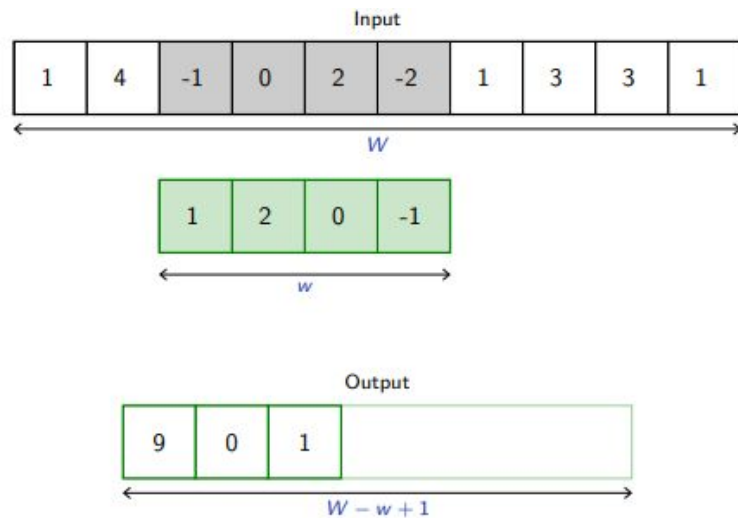
$$x = (x_1 \dots x_W) \quad u = (u_1 \dots u_w)$$

$$(x * u)[i] = \sum_{j=1}^W x_{i-1+j} u_j$$

Note: Mathematically one can rewrite the convolution operation similar to MLP where the effective weight matrix is no longer a dense matrix.

$$\mathbb{R}^D \rightarrow \mathbb{R}^{D'}; \quad x \mapsto \sigma(wx + b)$$

CNN [Convolution Operator]



CNN [Convolution Operator]

Idea: A transformation meaningful at a certain location can / should be used everywhere.

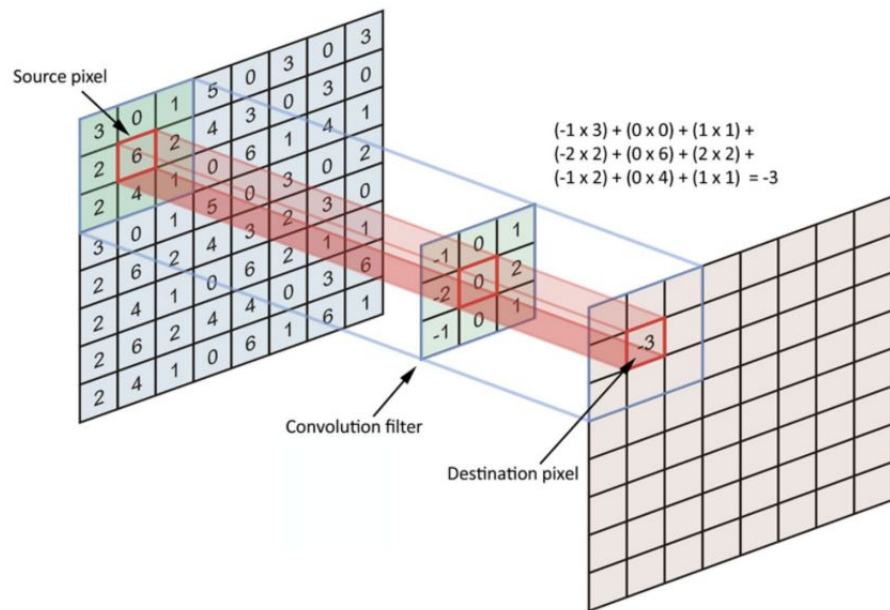
CNN applies the same linear **affine** transformation locally, everywhere. (Don't forget the additive bias)

Convolution in CNN differs from the usual convolution in classical signal processing. The operation is more of **correlation** than convolution.

Formally,

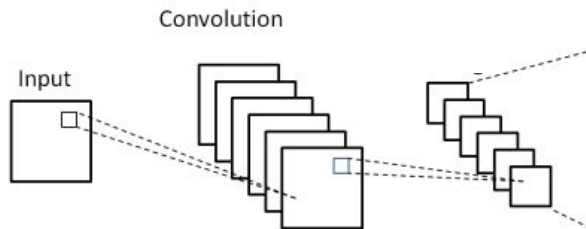
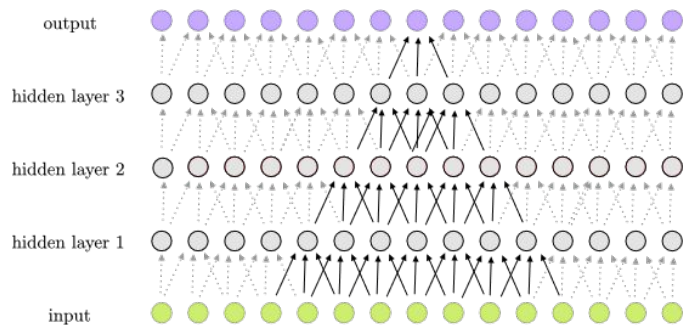
Convolution defines the relation between a system's input, output and impulse response. Correlation measures the similarity between two signals. In S&S similar equation is just a mathematical coincidence.

CNN [Convolution Operator]

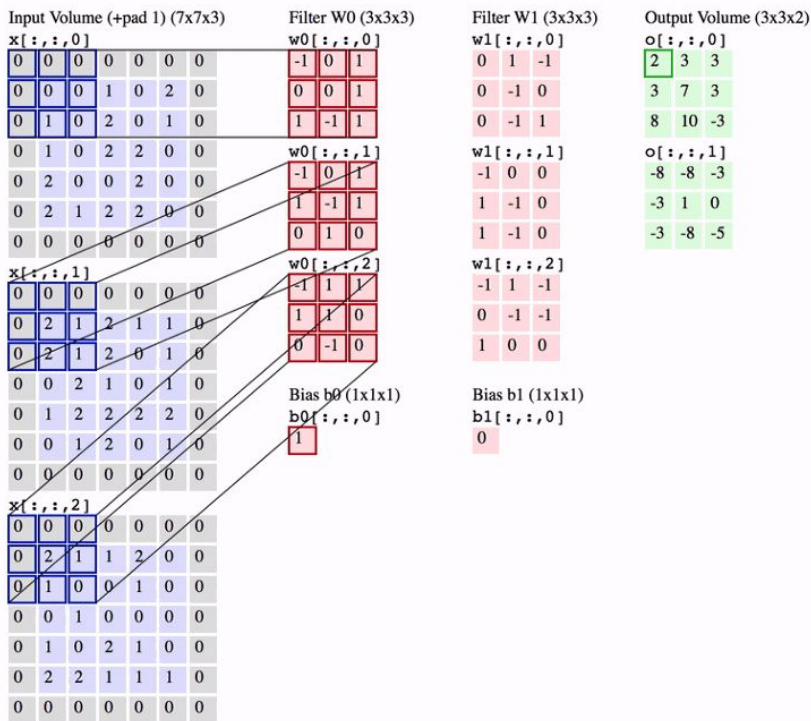


A channel generated by a convolution layer is called **activation map**.

A sub-area of input influencing a component of the output is called the **receptive field**.



CNN [Convolution Operator]



An RGB image is a multi-channel 2D signal. It can be viewed as three 2D signals, one for each color **red**, **green**, and **blue**, and is stored by convention in PyTorch in a tensor of shape **`torch.Size([3, H, W])`**

Input tensor (**C** × **H** × **W**)

kernel (**C** × **h** × **w**)

output (**H - h + 1**) × (**W - w + 1**)

D such convolutions are combined to generate output of shape

D × (**H - h + 1**) × (**W - w + 1**)

C: **C_{in}**

D: **C_{out}**

Classical Operators

Edge detection

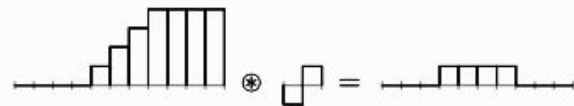
 * $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$ = 

Sharpen

 * $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ = 

Kernel

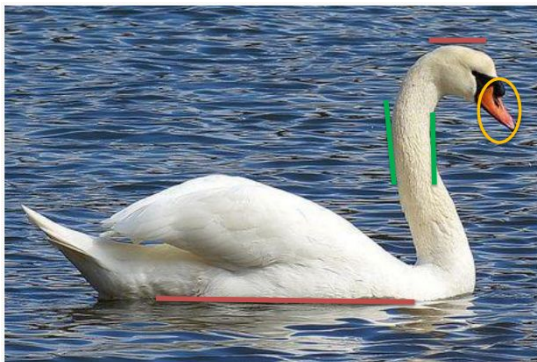
$$(0, 0, 0, 0, 1, 2, 3, 4, 4, 4, 4) \oplus (-1, 1) = (0, 0, 0, 1, 1, 1, 1, 0, 0, 0).$$



[DSP course]

[DIP course]

Back to CNNs



Feature 1: Horizontal Lines

Feature 2: Vertical Lines

Feature 3: Orange Ellipse

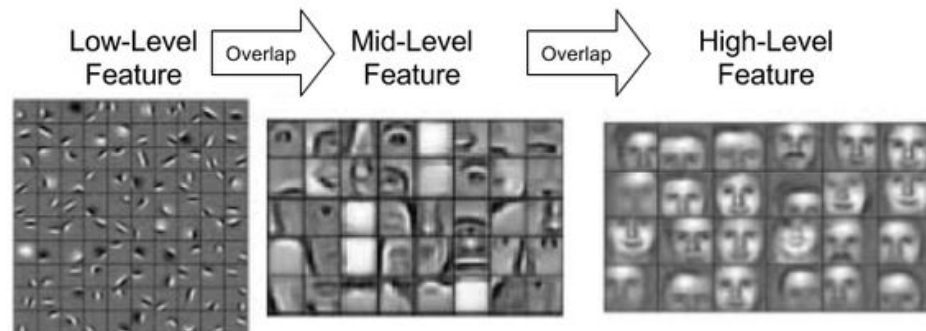
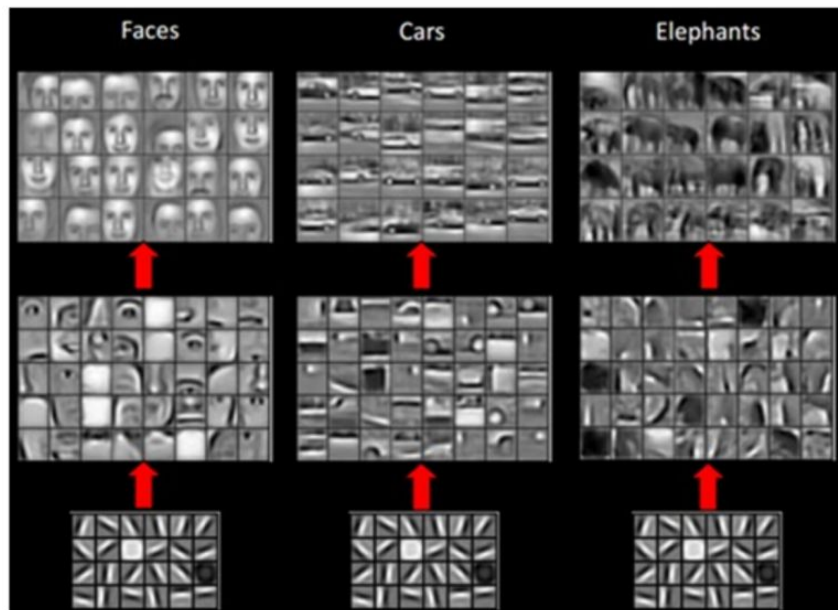


Depth:

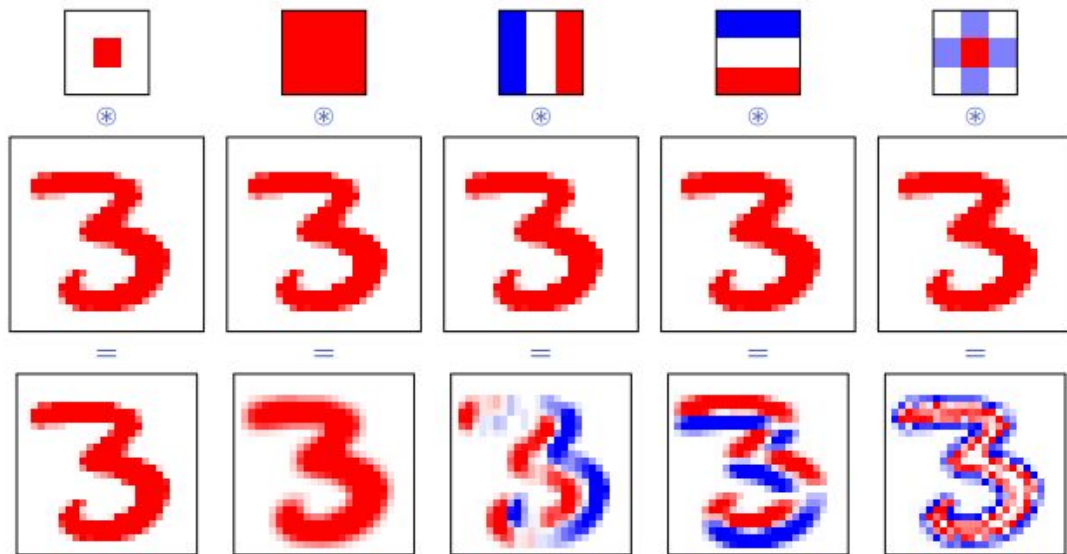
Combine vertical & horizontal lines to make diagonal lines.

Combine further to form shapes.

Back to CNNs



Back to CNNs



- positive values are in red
- negative values are in blue
- white is zero

1st filter keep the image unchanged, 2nd blurs it, the 3rd and 4th compute respectively vertical and horizontal edge responses, and 5th sharpens the image by keeping only high frequencies.

Padding, Stride, and Dilation

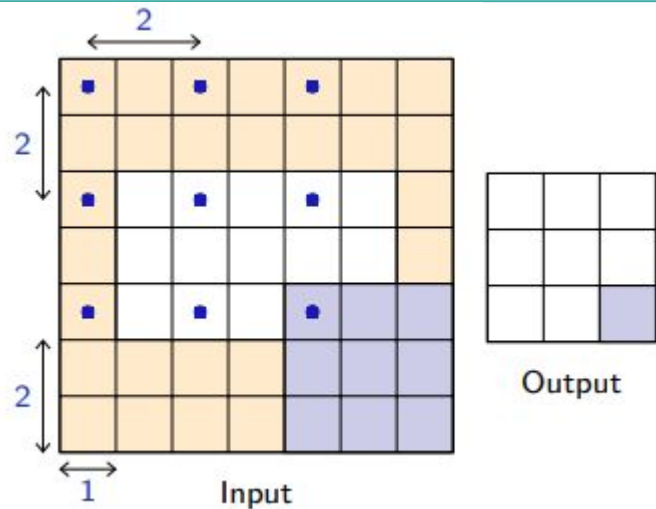
```
torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1,  
padding=0, dilation=1, groups=1, bias=True)
```

```
F.conv2d(input, weight, bias=None, stride=1, padding=0, dilation=1,  
groups=1)
```

Convolutions have three additional parameters:

- The padding specifies the size of a zeroed frame added around the input
- the stride specifies a step size when moving the kernel across the signal
- the dilation modulates the expansion of the filter without adding weights

Padding, Stride, and Dilation

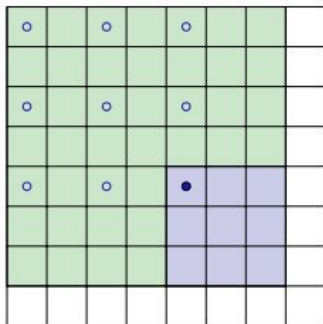


input of size (3×5) ,
padding of $(2, 1)$,
stride of $(2, 2)$,
kernel of size (3×3) ,
output is 3×3

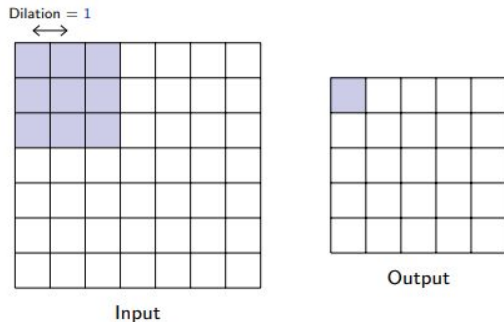
Padding $(2, 1)$ means two rows are added at the top/bottom and one column on the left/right

A convolution with a stride greater than 1 may not cover the input map entirely, hence may ignore some of the input values.

The green area shows which part of the input signal the output actually depends on.

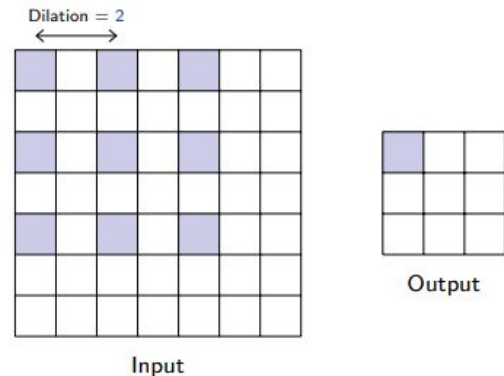


Padding, Stride, and Dilation



Dilation modulates the expansion of the filter support by adding rows and columns of zeros between coefficient.

The resulting operation can be envisioned as a convolution with a regularly sparsified filter.

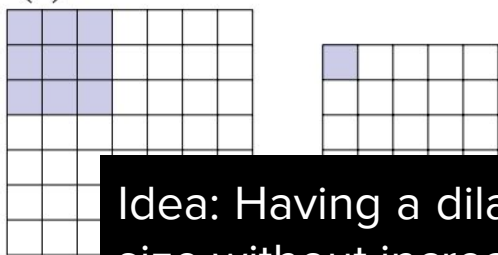


The output is smaller than with a standard convolution because the filter cannot be moved as much.

A 1D convolution with a kernel of size k and dilation d can be interpreted as a convolution with a filter of size $1 + (k - 1)d$ with only k non-zero coefficients.

Padding, Stride, and Dilation

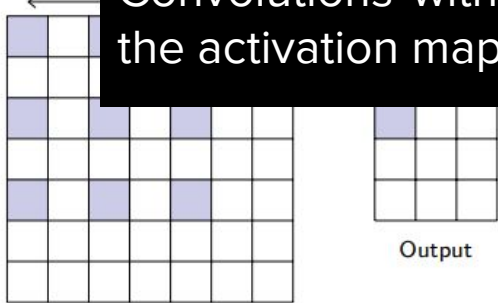
Dilation = 1



Dilation modulates the expansion of the filter support by adding rows and columns of zeros between coefficient

Idea: Having a dilation greater than one increases the receptive field size without increasing the number of parameters.

Dilation =



Convolutions with stride or dilation strictly greater than one reduce the activation map size.

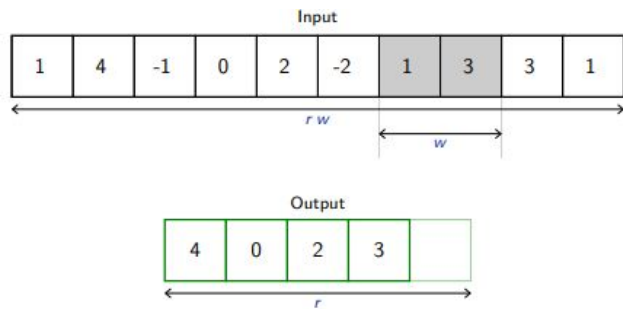
much.

Input

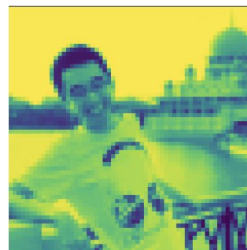
Output

A 1D convolution with a kernel of size k and dilation d can be interpreted as a convolution with a filter of size $1 + (k - 1)d$ with only k non-zero coefficients.

Pooling



original image



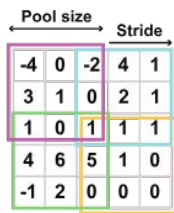
average pooling



max pooling

Pooling is used to extract low-dimensional representation by grouping.

Pooling provides invariance to any permutation or local translations.



Features

Max Pooling



Min Pooling



Output

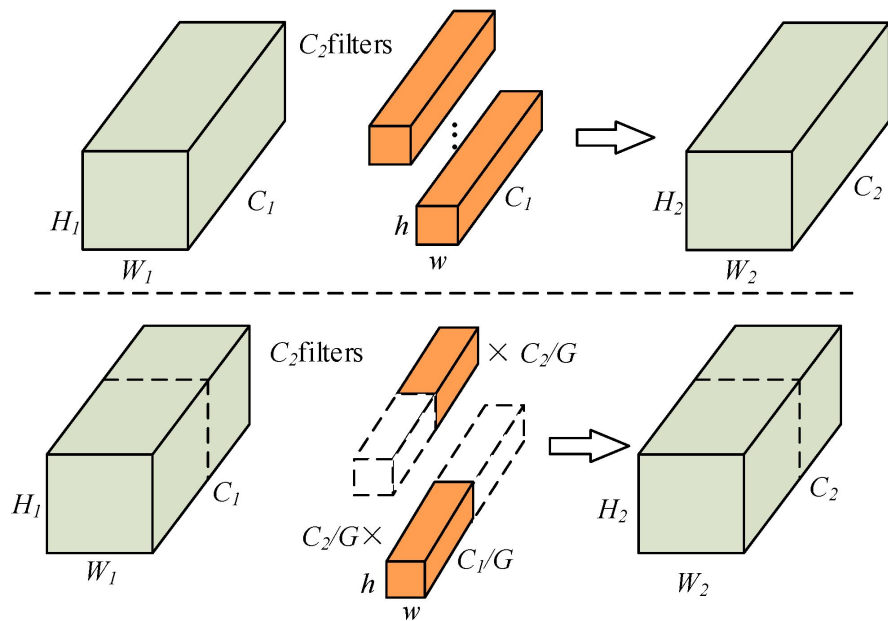
Average Pooling



```
F.max_pool2d(input, kernel_size, stride=None, padding=0, dilation=1, ceil_mode=False, return_indices=False)
```

```
torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)
```

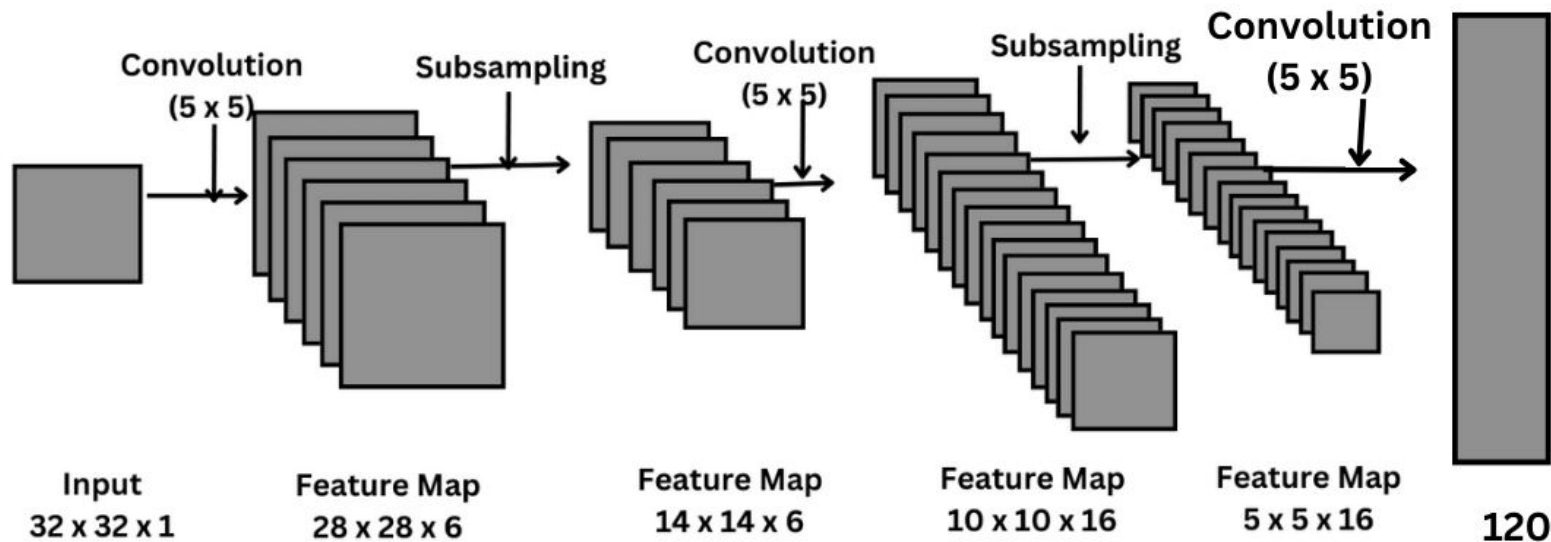
Group [Channel Splitting]



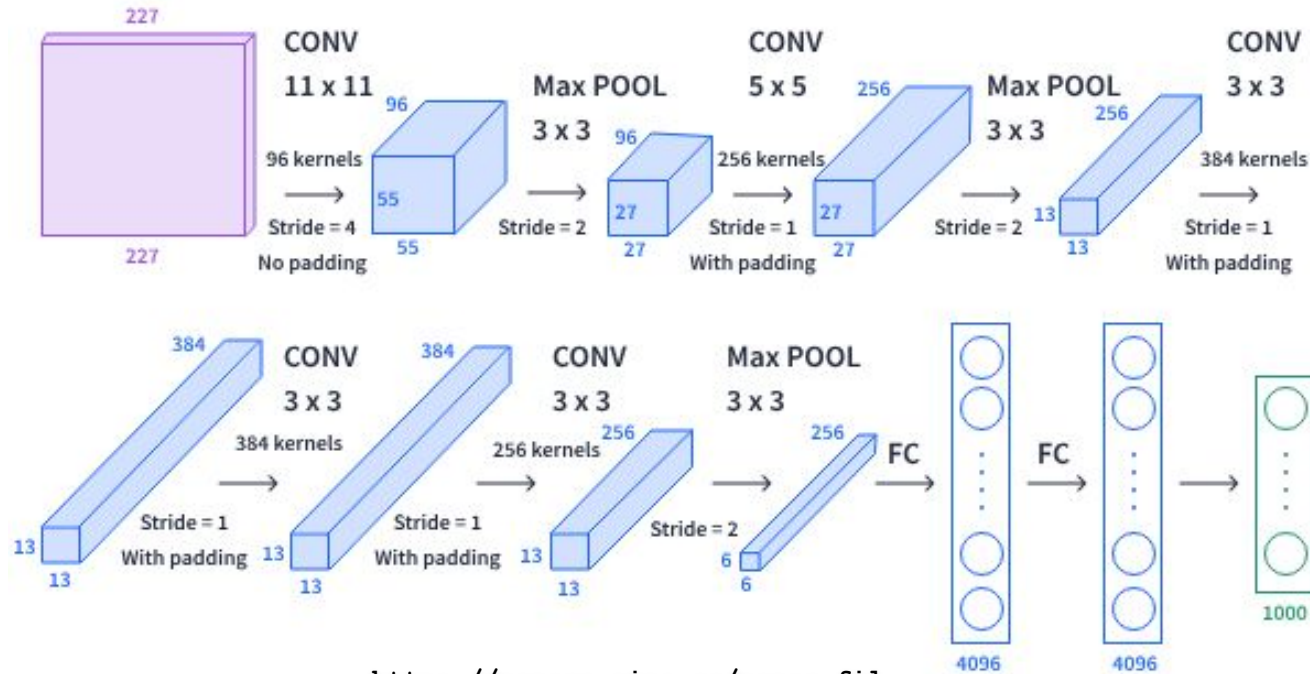
Group Convolution - split channels equally and perform regular convolution for each group.

C_{in}/C_{out} should be divisible by Group size.

CNN Architectures [LeNet]

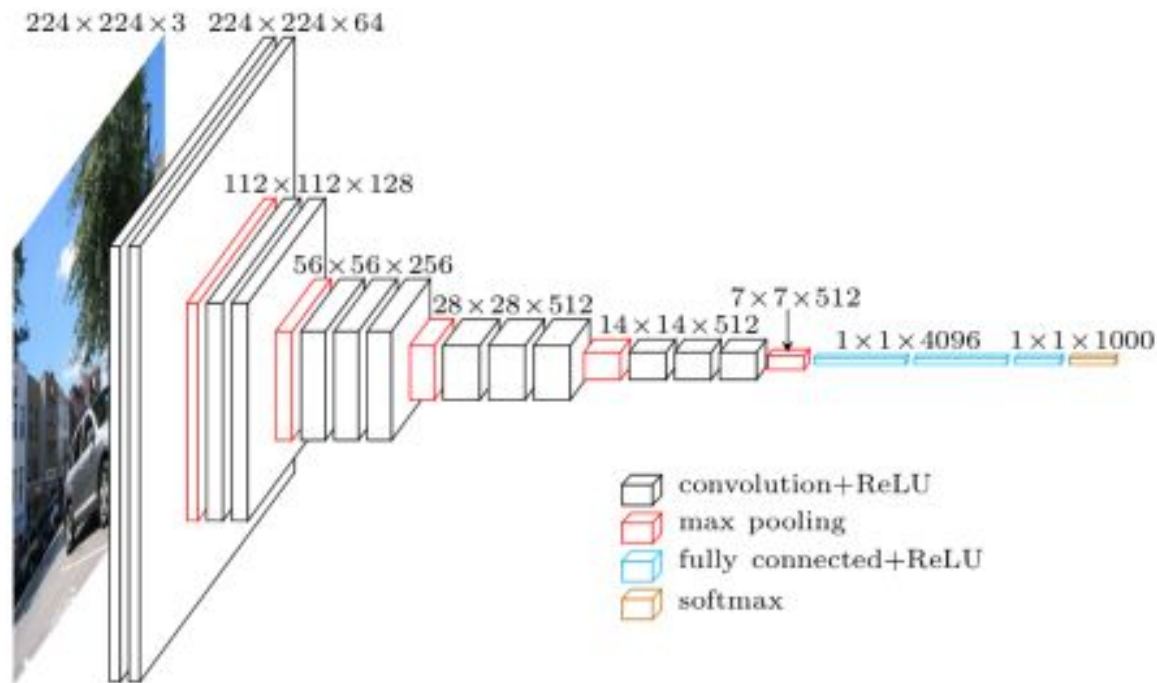


CNN Architectures [AlexNet]

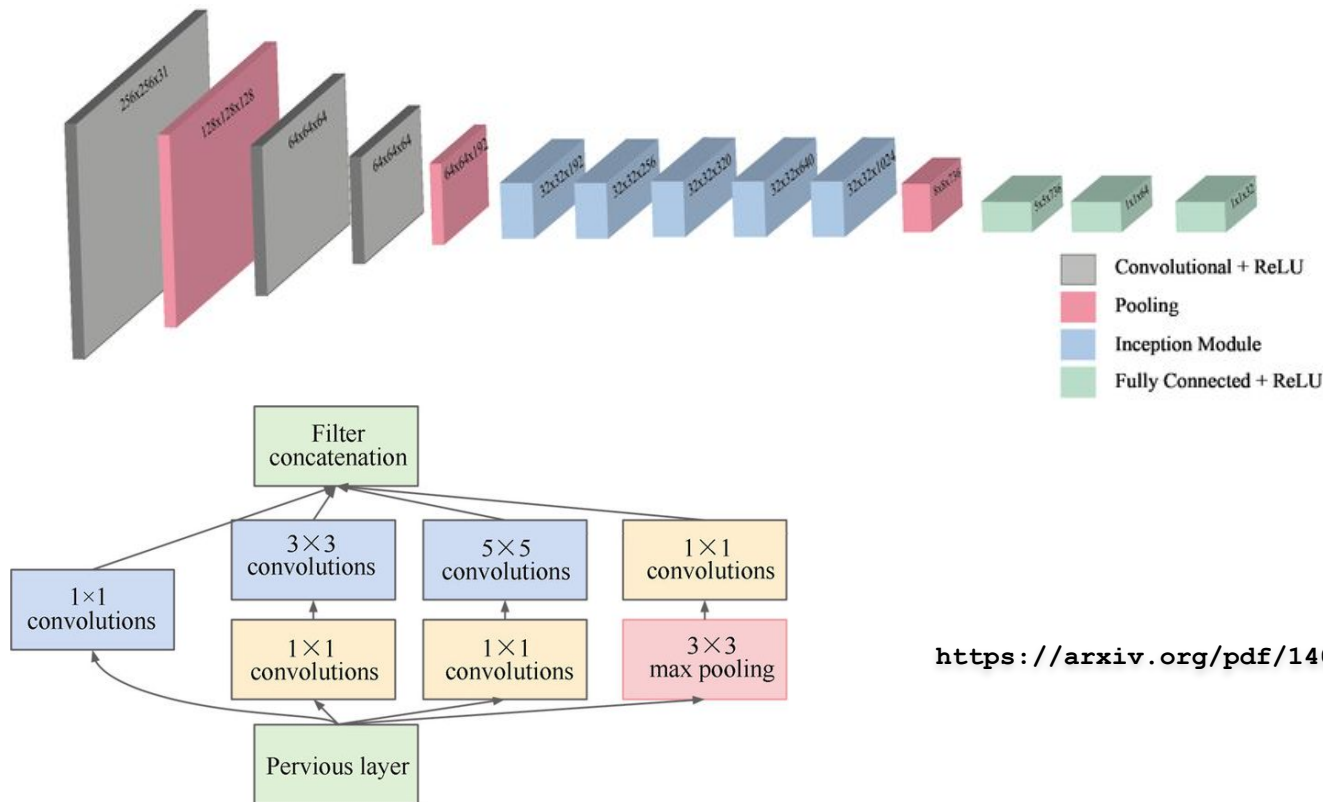


https://papers.nips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

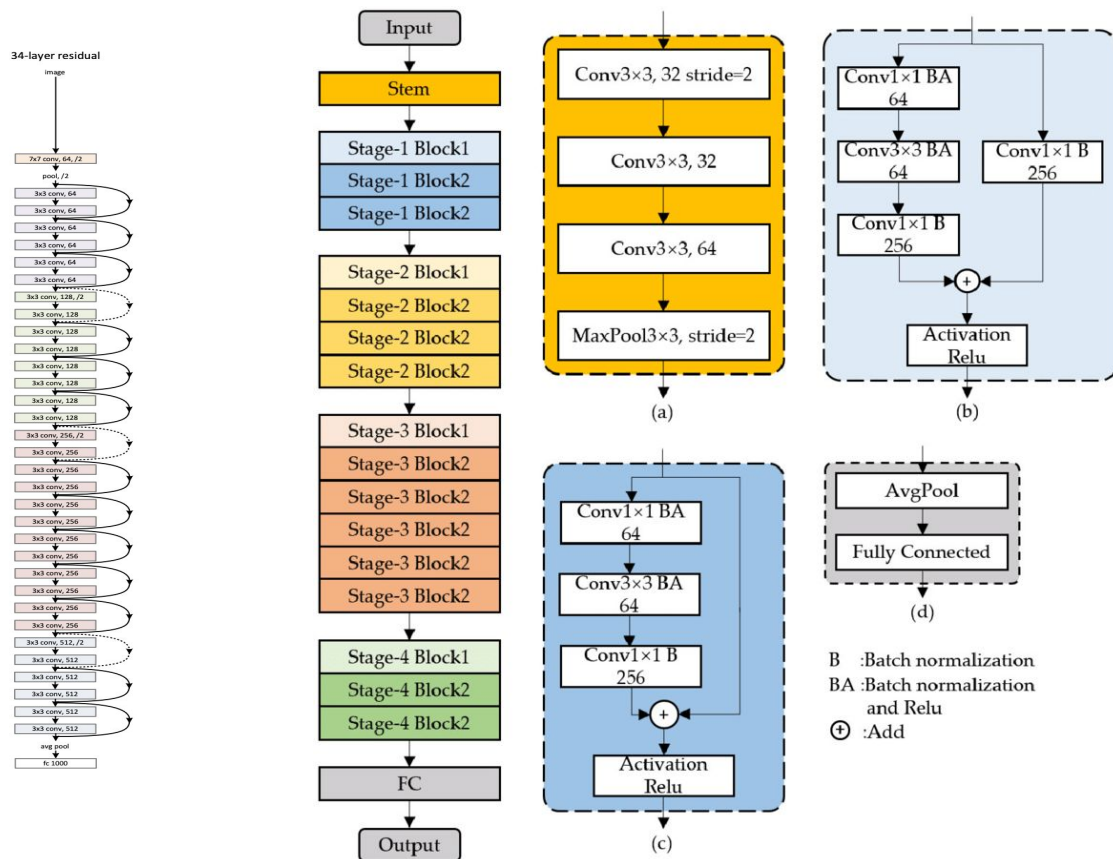
CNN Architectures [VGG]



CNN Architectures [GoNet-Inception]



CNN Architectures [Residual Net]



https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

CNN Architectures

Practice Homework:

Starting with simple feed-forward CNNs try and code various CNN architectures from scratch and compare the training time between your and PyTorch default implementation on CIFAR-10 Dataset.

```
https://pytorch.org/vision/stable/models.html  
model = torchvision.models.resnet50(weights=None)
```

Don't Use pre-trained weights

Old weights with accuracy 76.130%

```
resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)
```

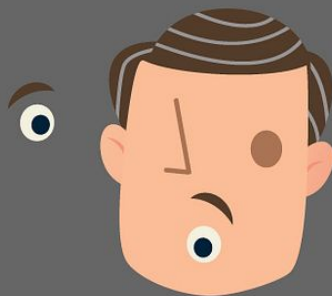
New weights with accuracy 80.858%

```
resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
```

Thanks



IT'S A FACE!



IT'S A FACE!



HMMM, I'M NOT SURE.